

Progressive Reinforcement Learning for Paddle Duel

Combined Project Report: Tabular Control to Pixel-Based DQN and Self-Play

Course: 51.512 Reinforcement Learning for Embodied AI

Academic Year: 2025-2026

Submission date: 8 August 2026

Name	Student ID	Primary contribution
Duan Xu	1010728	Level 5: fixed-opponent baselines, self-play with opponent pool, full-scale training/evaluation and competition checkpoints.
Manan Bimal Mehta	1010949	Levels 1-3 tabular RL and Level 4 Feature-DQN/CNN-DQN, including training, evaluation and baseline comparisons.
Zheng Chengsheng	1010719	Final report consolidation, metric cross-checking and reproducibility documentation.

Full code, notebooks and saved checkpoints: <https://github.com/FlyGCB/RL-Project>

1. Introduction

This project studies reinforcement learning for Paddle Duel, a simplified Pong-style one-rally control task. The five levels form a progressive experimental storyline: classical tabular temporal-difference control, exploration design against a scripted opponent, robustness across an opponent zoo, pixel-based deep Q-learning, and finally two-agent self-play. The environment itself was provided and was not modified.

Each player chooses UP, DOWN or STAY. A rally ends when a side scores or when the time limit is reached. The official reward includes the scoring reward, small paddle-hit bonuses, and a -0.001 movement penalty for UP/DOWN. Levels 1-3 use a seven-element discretized state; Level 4 moves to a 150x320x3 RGB observation; Level 5 trains learned agents against learned opponents.

The project follows a common BaseAgent interface, separates training from evaluation, saves checkpoints for reload without retraining, uses held-out evaluation seeds, compares against Random and Tracking baselines where appropriate, and reports learning curves, replay evidence, failure cases and limitations.

2. Experimental methodology

Across the project, training generally used seed 0 with episode-dependent offsets and evaluation used unseen seeds beginning at 10,000. Final evaluations for Levels 1-4 typically used 200 episodes with exploration disabled. In addition to reward and win rate, diagnostics include paddle hits, episode length, movement/STAY counts, miss rate, timeout rate and replay trajectories. Tabular experiments used $\alpha=0.1$ and $\gamma=0.99$, usually for 2,000 episodes with a 400-episode fully random warm-up. Neural Level 4 experiments also used 2,000 episodes.

3. Level 1 - Tabular control, solo vs wall

Level 1 asks which classical tabular TD-control method gives the best behaviour in the simplest setting. A shared TabularControlAgent implements Q-learning, SARSA and Expected SARSA, differing only in the bootstrap target. The seven-dimensional state contains controlled-paddle y-bin, other-paddle y-bin, ball x/y bins, horizontal/vertical velocity signs and speed bin.

Two comparison passes were performed. The improved diagnostic run used 2,000 episodes, a 400-episode forced-random warm-up, $\alpha=0.1$, $\gamma=0.99$, $\epsilon_{\text{start}}=1.0$, $\epsilon_{\text{min}}=0.05$ and $\epsilon_{\text{decay}}=0.9999$. An ablation masked the -0.001 movement penalty from the training TD target only, while official reward remained the evaluation metric. This was intended to test whether the movement cost was prematurely encouraging STAY, and it should be treated transparently as reward shaping rather than as an unchanged official-reward run.

Agent	Mean reward	Std reward	Mean hits	Mean steps	Learned states
Q-learning	-0.0445	0.7053	2.57	437.0	33,082
SARSA	-0.0864	0.3870	1.81	336.4	32,872
Expected SARSA	-0.1202	0.1878	2.72	465.7	32,336

Q-learning achieved the least-negative mean reward, Expected SARSA produced the most hits and longest episodes and had the lowest reward variance, while SARSA moved less. All three policies remained far from solving the task after 2,000 episodes. Replay diagnostics confirmed that the paddle genuinely moved; STAY was the single most common action but not a majority of all actions. A major limitation is that the reported Level 1 training result comes from a single training seed.

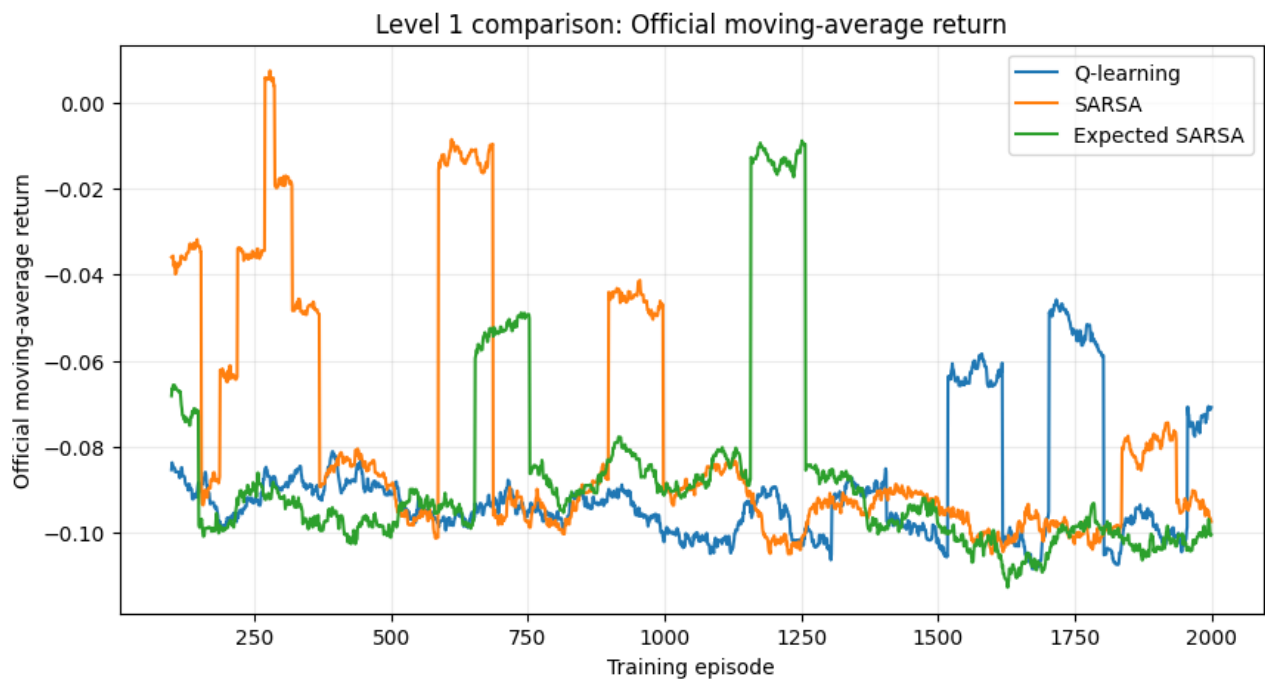


Figure 1. Level 1 official moving-average return during training.

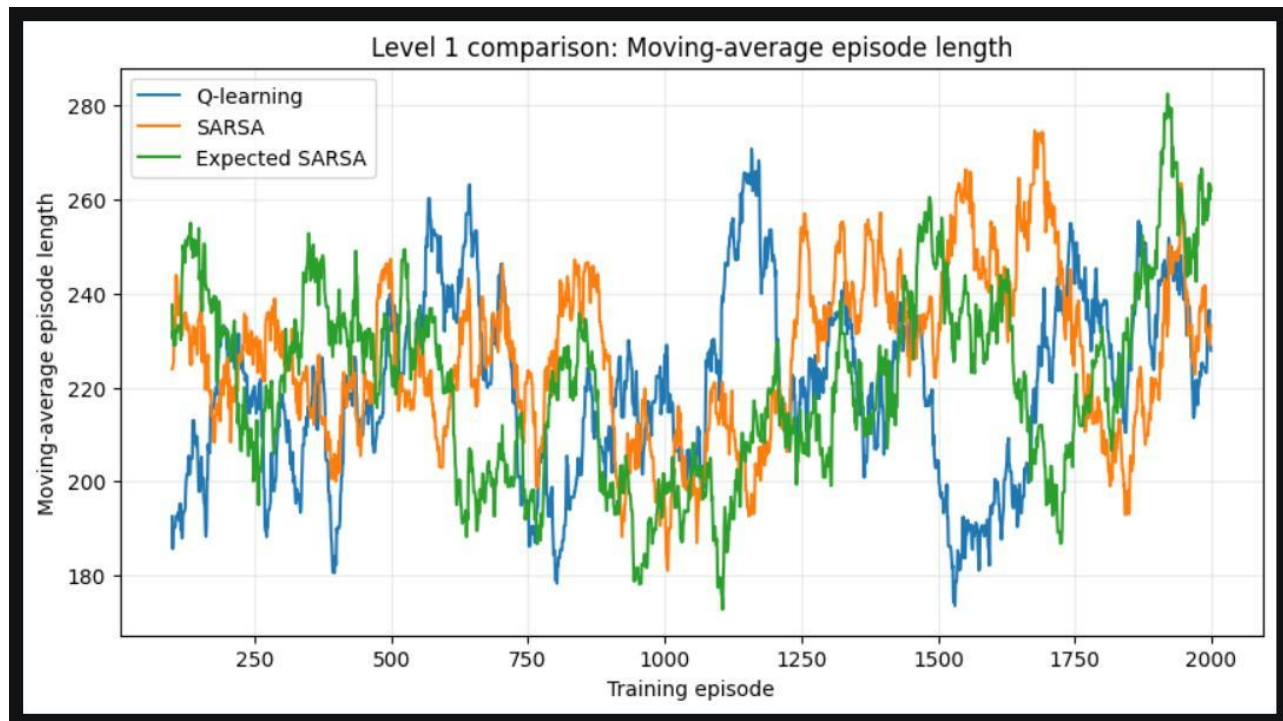


Figure 2. Level 1 moving-average episode length.

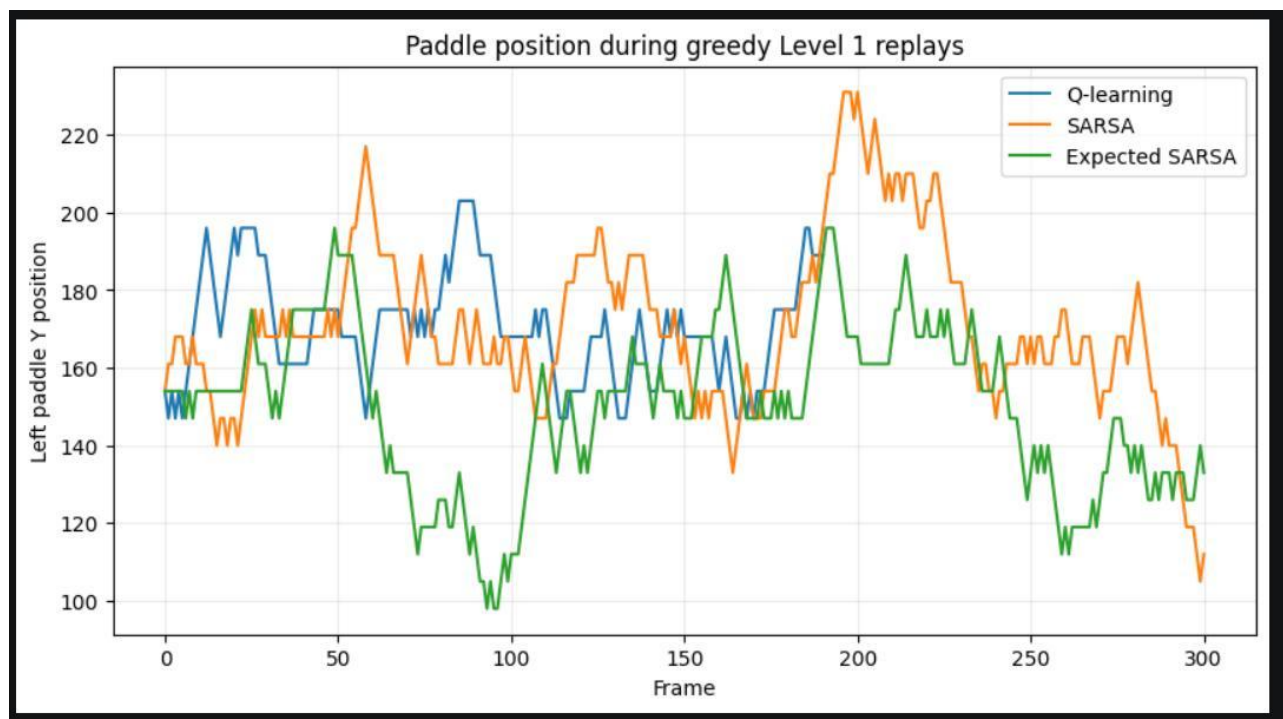


Figure 3. Paddle position during greedy Level 1 replay.

4. Level 2 - Tabular control vs a scripted opponent

Level 2 introduces a scripted opponent while retaining the same seven-dimensional tabular state. The first controlled comparison isolates the TD target by training Q-learning and SARSA under the same 2,000-episode budget and 400-episode forced-random phase.

Agent	Mean reward	Win rate	Mean hits	Mean steps	Miss rate
Q-learning	2.1649	21.5%	1.11	191.50	78.5%
SARSA	1.8624	18.5%	0.94	170.83	81.5%
Random baseline	1.5015	15.5%	0.77	141.75	84.5%
Tracking baseline	7.6266	75.5%	7.98	741.79	24.0%

Q-learning outperformed SARSA and the Random baseline on the principal metrics, but both learned agents remained far below the scripted Tracking Agent. This motivated fixing the algorithm to Q-learning for the exploration-schedule experiment.

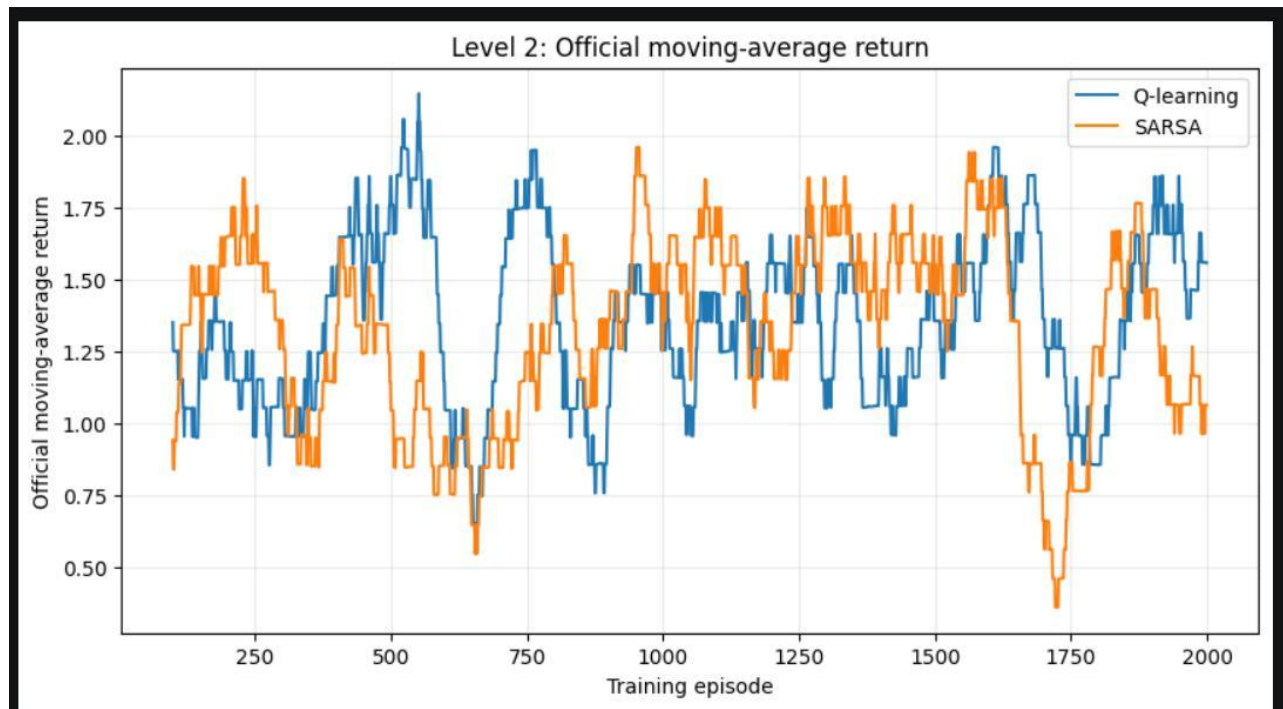


Figure 4. Level 2 official moving-average return: Q-learning vs SARSA.

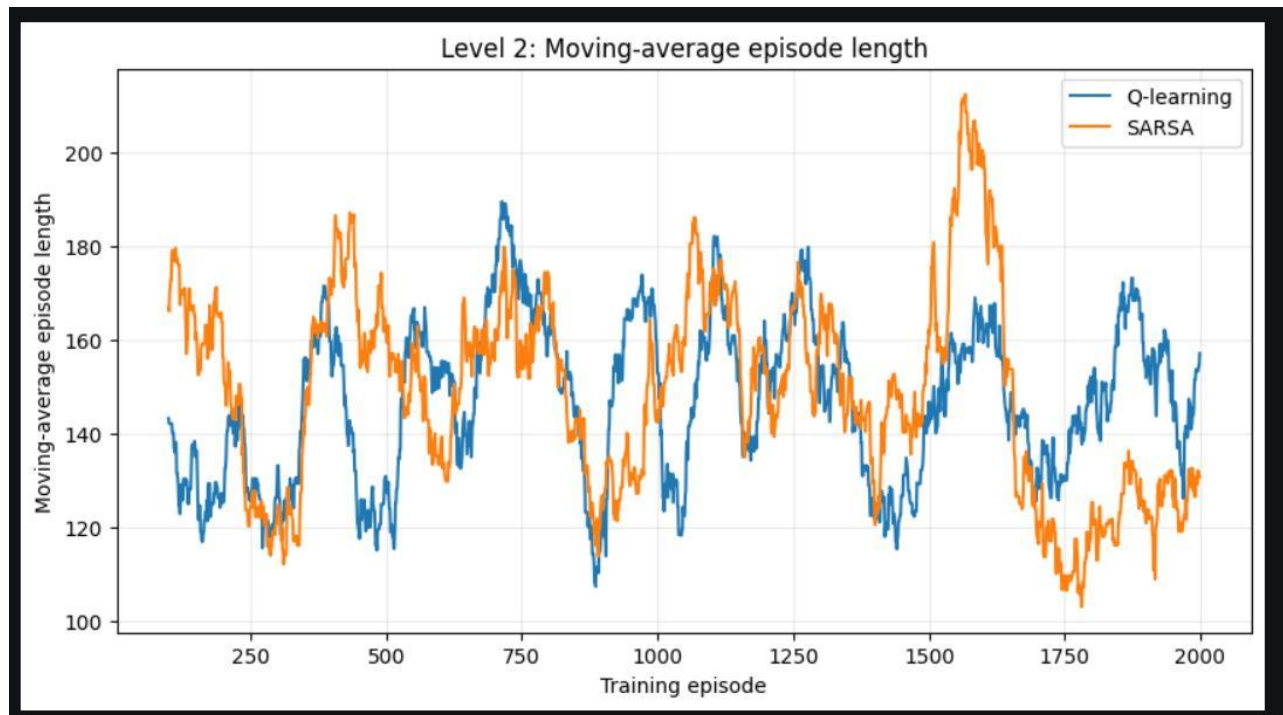


Figure 5. Level 2 moving-average episode length.

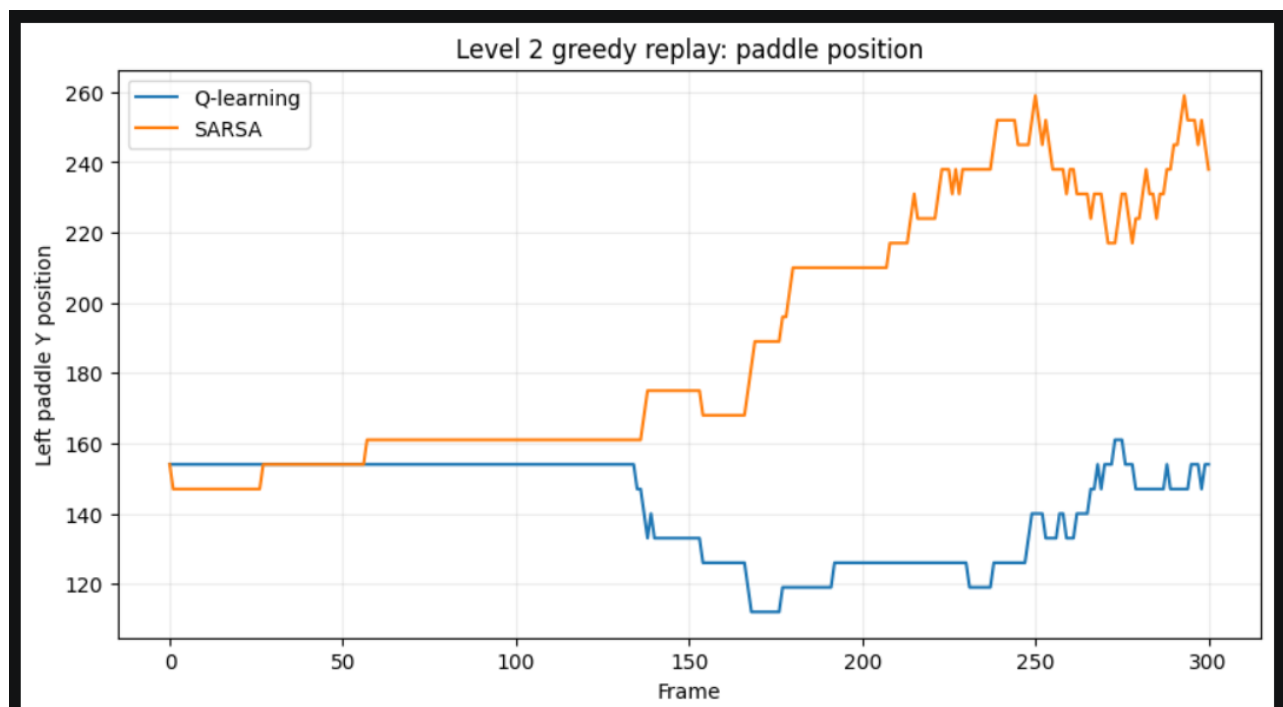


Figure 6. Paddle position during greedy Level 2 replay.

4.1 Exploration schedule comparison

With Q-learning fixed, three epsilon schedules were compared after the same 400-episode random warm-up: constant epsilon=0.20, linear decay from 1.00 to 0.05 and exponential decay from 1.00 to 0.05.

Schedule	Mean reward	Win rate	Mean hits	Mean steps	Move actions
Constant	1.6729	16.5%	1.07	191.07	41.32
Linear	2.0746	20.5%	1.245	224.62	50.14
Exponential	1.8778	18.5%	1.025	186.89	33.75

Linear decay gave the best aggregate evaluation performance. Constant epsilon keeps injecting random actions late in training, while the tested exponential schedule reduced exploration more aggressively; the linear schedule provided the strongest balance in this run. A single exponential-decay replay won quickly through favourable early positioning, illustrating why individual replays should not replace multi-episode evaluation.

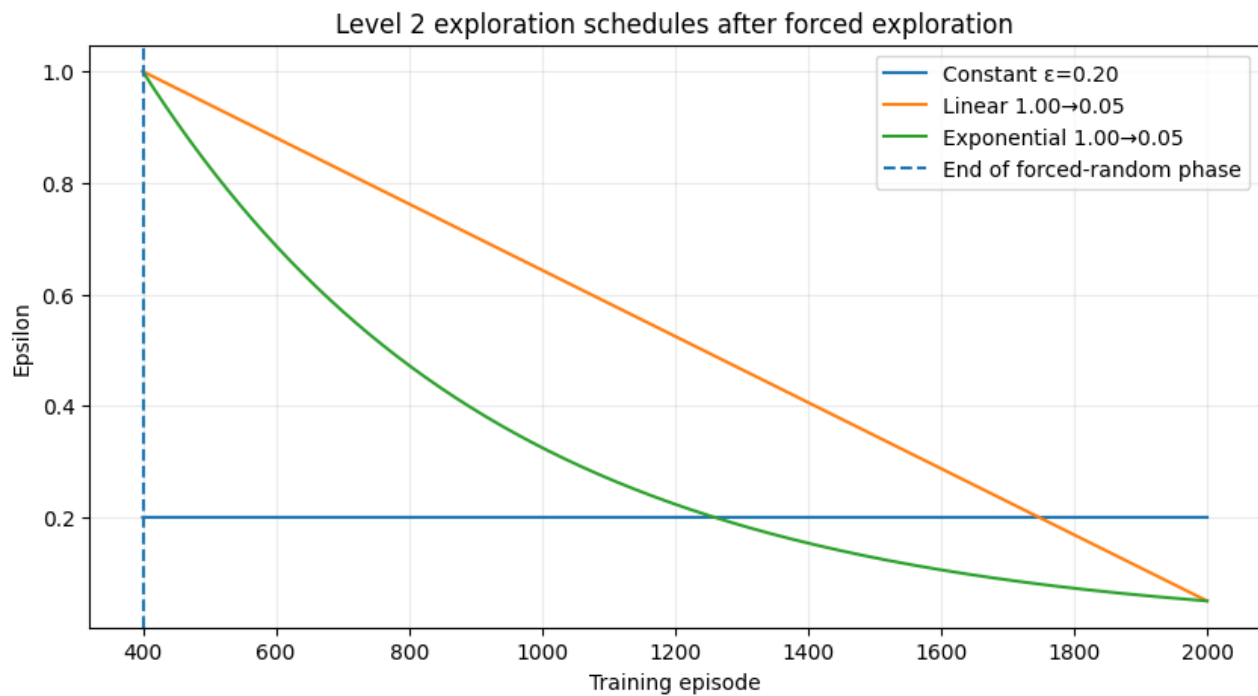


Figure 7. Epsilon schedules used in the Level 2 exploration experiment.

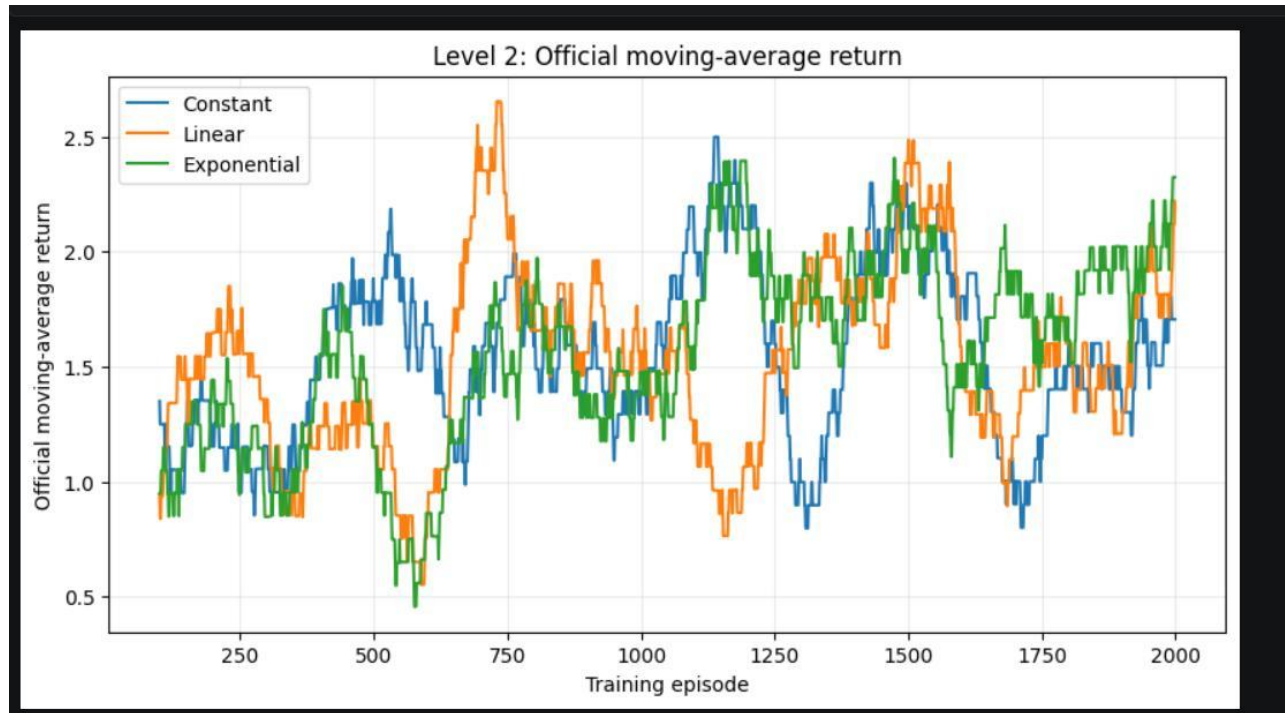


Figure 8. Level 2 moving-average return for the exploration-schedule comparison.

5. Level 3 - Robustness across an opponent zoo

Level 3 tests generalization rather than specialization. A single Expected SARSA agent was trained for 2,000 episodes directly against the mixed zoo opponent and then evaluated separately against random, weak, tracking, strong and mixed-zoo opponents. Hyperparameters were $\alpha=0.1$, $\gamma=0.99$, $\epsilon_{\text{start}}=1.0$, $\epsilon_{\text{min}}=0.05$, $\epsilon_{\text{decay}}=0.9999$ with a 400-episode warm-up. The learned Q-table contained 77,102 states.

Opponent	Mean reward	Win rate	Mean hits	Mean steps	Miss rate
Random	2.4932	25.0%	0.60	114.99	75.0%
Weak	1.5004	15.0%	0.88	163.13	85.0%
Tracking	0.8090	8.0%	1.13	195.68	92.0%
Strong	0.6597	6.5%	1.20	206.92	93.5%
Zoo	1.7062	17.0%	0.87	158.07	83.0%

Average win rate across opponents was 14.3%, with a worst-case 6.5% against Strong. Stronger opponents produced more hits and longer episodes because they returned the ball more consistently, yet they still won more often. The strong-opponent replay shows a clean failure mode: the agent barely reacts before conceding, suggesting that the zoo-trained tabular policy lacks sufficiently fast and precise defence.

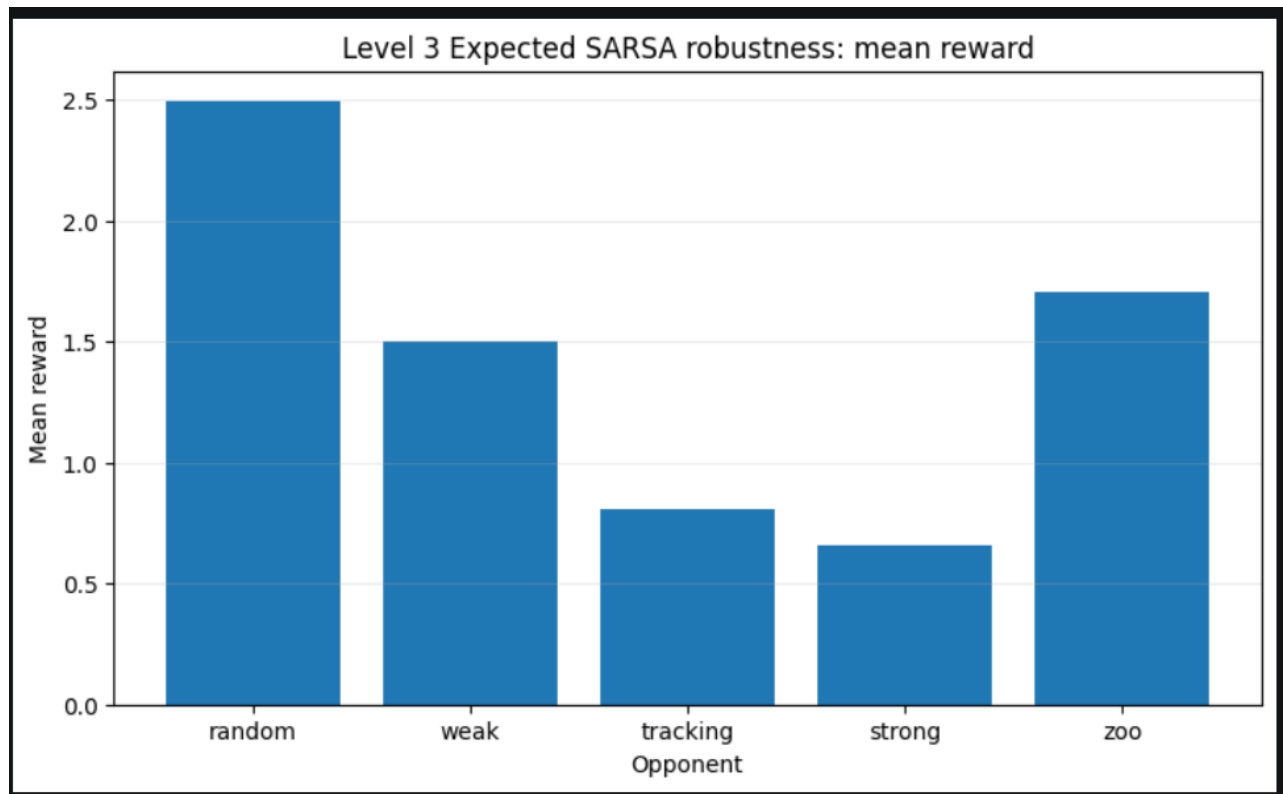


Figure 9. Level 3 mean reward across opponent types.

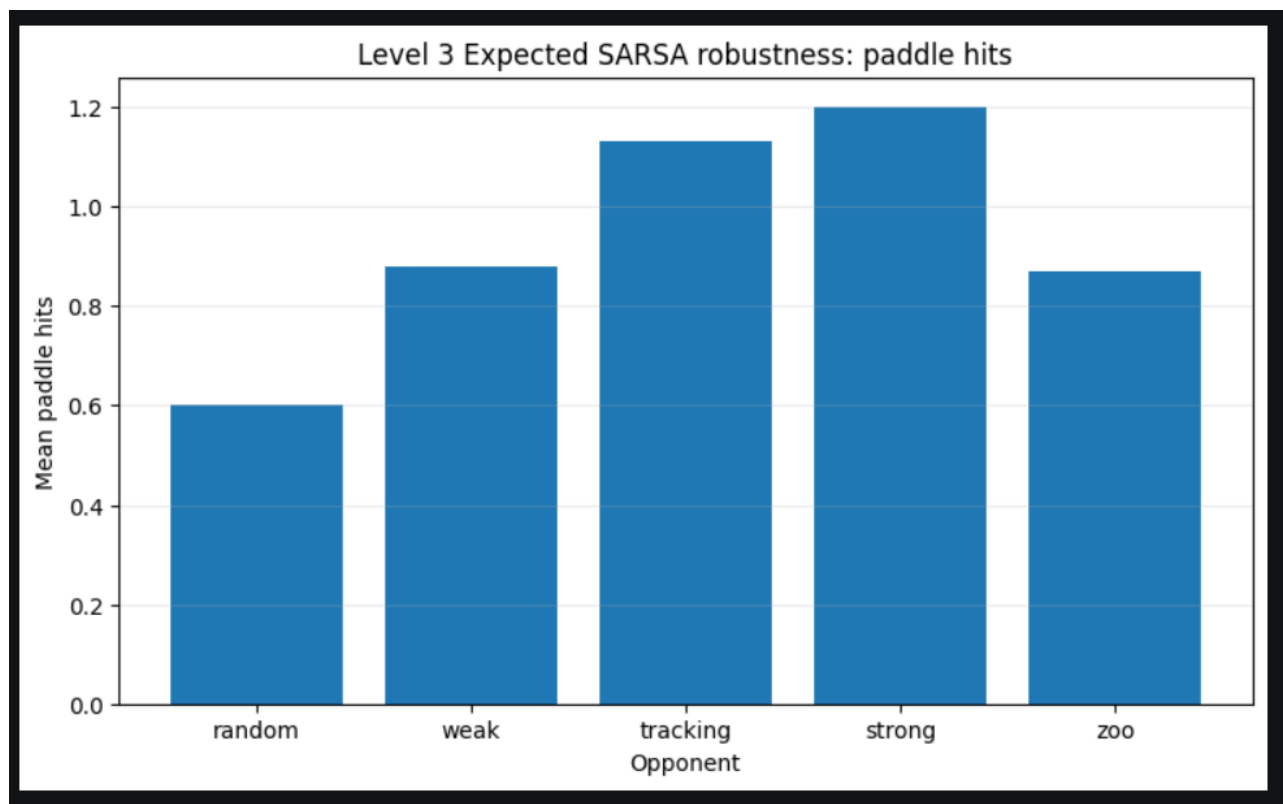


Figure 10. Level 3 mean paddle hits across opponent types.

6. Level 4 - Pixel-only observation and deep Q-learning

Level 4 replaces the compact tabular observation with pixels. Two DQN variants were deliberately built so that the effect of representation could be studied: Feature-DQN reconstructs a compact coordinate state from the image, while CNN-DQN learns a visual representation directly from stacked frames. Both use experience replay, a target network, Adam, Huber loss and gradient clipping.

6.1 Feature-DQN

Feature-DQN extracts approximate x/y coordinates of both paddles and the ball from each RGB frame and concatenates frame-to-frame deltas, yielding a 12-dimensional feature vector. An MLP maps 12 -> 128 -> 128 -> 3 Q-values. Training used $lr=1e-4$, $\gamma=0.99$, $\epsilon_{decay}=0.9999$, replay capacity 50,000, batch size 64, minimum buffer 2,000, target update every 1,000 steps and optimization every four steps. The 2,000-episode run took 2441.9 s on an NVIDIA RTX 5050 laptop GPU.

Agent	Mean reward	Win rate	Mean hits	Mean steps
Feature-DQN	1.9836	20.5%	1.075	172.2
Random Agent	1.5015	15.5%	0.77	141.8
Tracking Agent	7.6266	75.5%	7.98	741.8

Feature-DQN modestly improved over Random but remained far below Tracking and close to Level 2 tabular performance. Its failure replay showed heavy, erratic movement (107 UP, 24 DOWN, 16 STAY) for only one hit, suggesting movement without stable control.

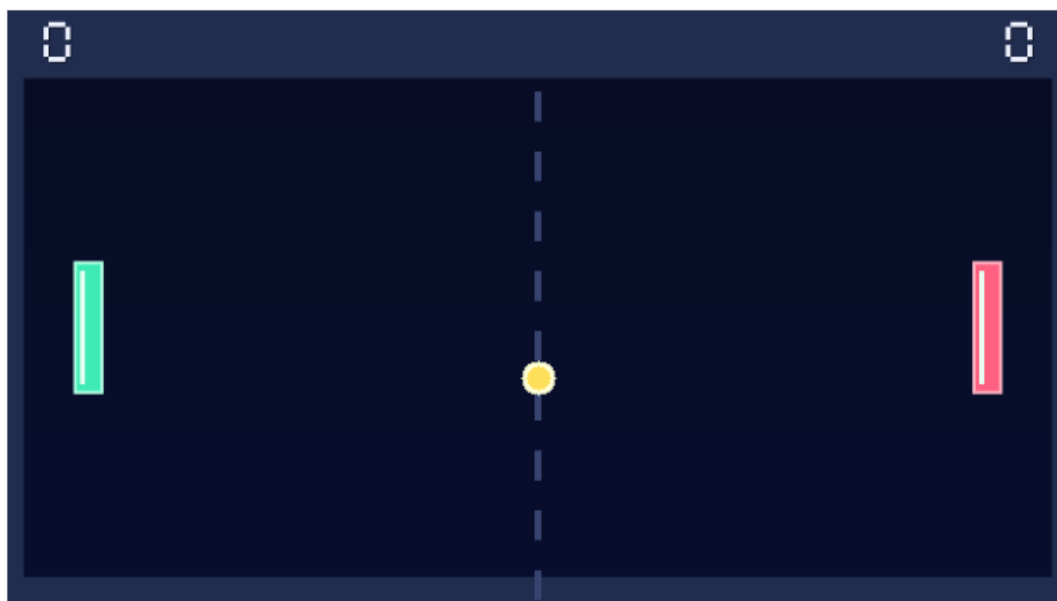


Figure 11. Level 4 RGB observation and engineered object-coordinate setting used by Feature-DQN.

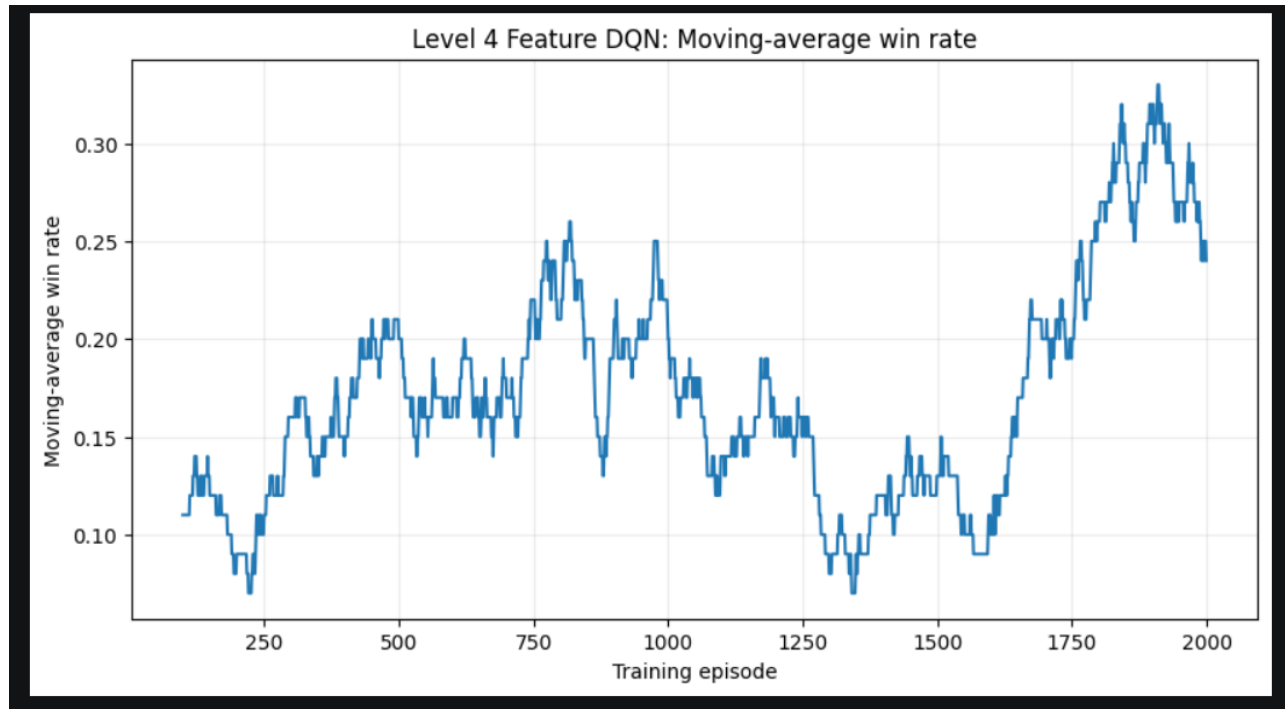


Figure 12. Representative Feature-DQN training curve.

6.2 CNN-DQN

CNN-DQN removes explicit coordinate extraction. Each frame is converted to grayscale, resized to 84x84 and stacked four frames deep. The Atari-style network uses Conv(4→32, 8x8 stride 4), Conv(32→64, 4x4 stride 2), Conv(64→64, 3x3 stride 1), a 512-unit fully connected layer and three Q-value outputs. Replay capacity was reduced to 30,000 and epsilon decay slowed to 0.99995. Training took 6992.4 s (about 1 h 56 min), roughly 2.9x Feature-DQN.

Agent	Mean reward	Win rate	Mean hits	Mean steps
CNN-DQN	6.9040	70.0%	3.67	482.4
Feature-DQN	1.9836	20.5%	1.075	172.2
Random Agent	1.5015	15.5%	0.77	141.8
Tracking Agent	7.6266	75.5%	7.98	741.8

CNN-DQN was the clearest improvement in Levels 1-4. Its 70% win rate was 49.5 percentage points above Feature-DQN and only 5.5 points below Tracking. The training curve remained modest until roughly episode 800 and improved sharply after about episode 1300, consistent with the additional difficulty of learning perception and control jointly. Reloading the saved checkpoint gave a 72% win rate in a separate 50-episode check. The seed-22 replay was also qualitatively stronger: CNN-DQN won with 44 UP, 32 DOWN and 62 STAY actions.

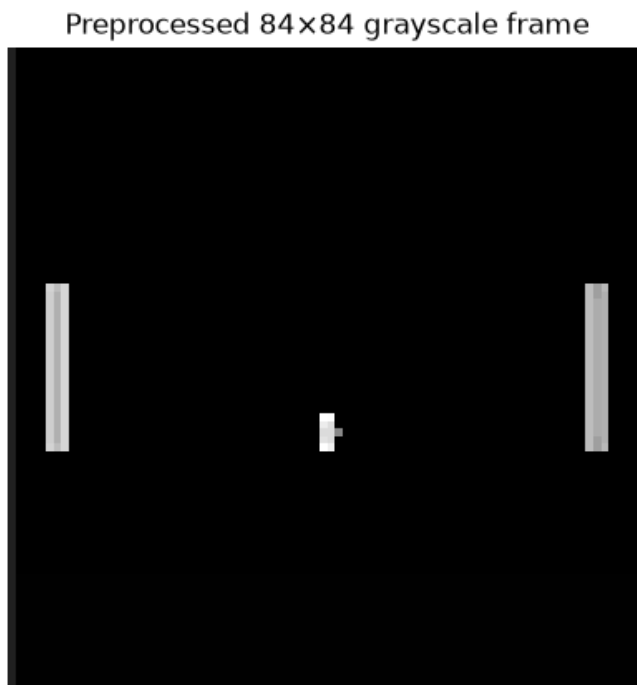


Figure 13. CNN-DQN preprocessing: grayscale 84x84 frame used in a four-frame stack.

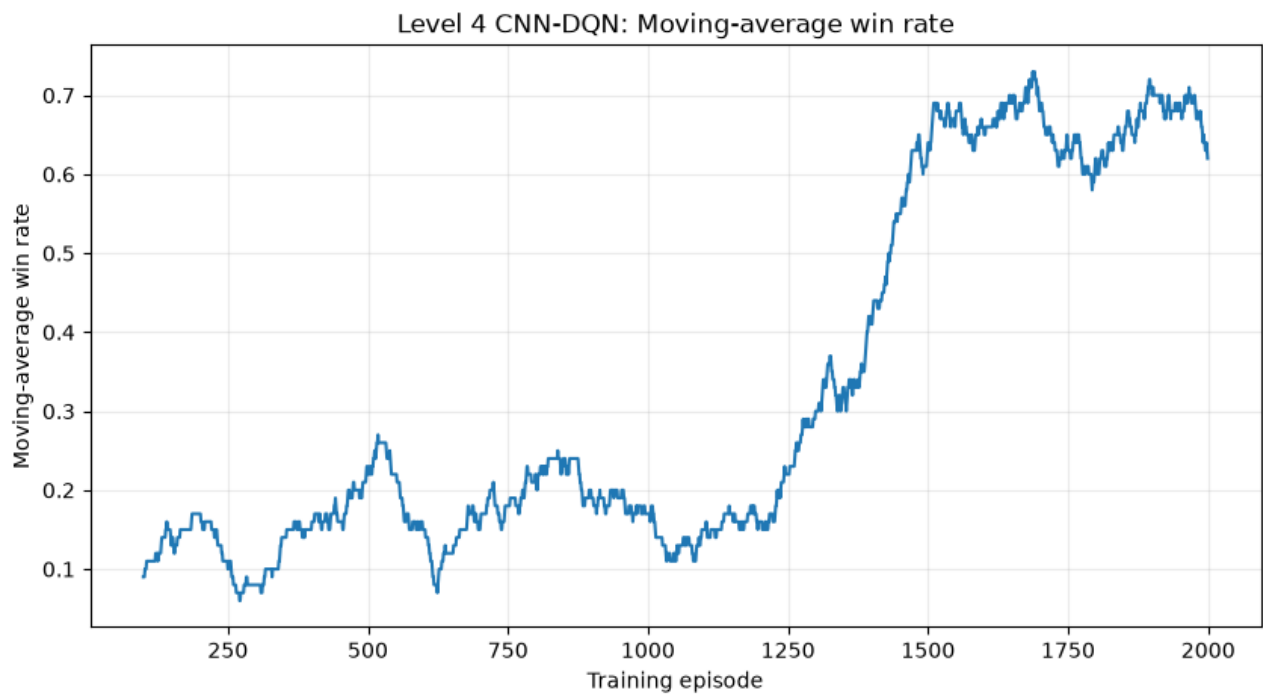


Figure 14. Representative CNN-DQN training curve.

7. Level 5 - Two learned agents and self-play

Level 5 uses learned agents on both sides and builds directly on the Level 4 Feature-DQN and CNN-DQN checkpoints. Both architectures follow the same two-stage design. Stage 5A is a fixed-opponent baseline: a fresh left learner trains against the frozen matching Level 4 model on the right, validating the two-agent training loop. Stage 5B warm-starts from 5A and introduces self-play with an opponent pool.

In 5B, the right-side opponent is a live mirror of the current learner with probability 0.8 and, with probability 0.2, a frozen snapshot sampled from an eight-slot FIFO opponent pool. Slot 0 remains pinned to the original frozen Level 4 model, while later slots are populated by snapshots every 200 episodes. This design is intended to reduce self-play cycling and catastrophic forgetting. Epsilon reopens to 0.3 on warm start rather than returning to 1.0.

Both stages and both architectures used 1,500 training episodes with level=5, render_scale=2 and max_seconds=15 during training; evaluation used max_seconds=30. Full-scale runs took about 80 minutes for Feature-DQN and 160 minutes for CNN-DQN on an RTX 3070 Ti. The final reported Level 5 numbers come from the real non-interactive full training logs; metrics not preserved in those logs are not reconstructed.

7.1 Final evaluation - 5A vs 5B

The final benchmark used 50 evaluation episodes against frozen Level 4, Random and Tracking. Tracking was placed on the left because it is side-aware and reads unmirrored information.

Feature-DQN opponent	5A win rate	5B self-play win rate
Frozen L4	54%	24%
Random	48%	28%
Tracking	2%	2%
5B vs 5A head-to-head	-	5B wins 44%
5B self-mirror	-	20% / 80% split

CNN-DQN opponent	5A win rate	5B self-play win rate
Frozen L4	20%	72%
Random	46%	98%
Tracking	2%	20%
5B vs 5A head-to-head	-	5B wins 86%
5B self-mirror	-	42% / 58% split

Self-play produced opposite outcomes for the two architectures. Feature-DQN 5B peaked around 81% training-time win rate near episode 500, then declined to about 27% by episode 1400 and ended around 30%. Its reward followed the same pattern, indicating a non-stationarity failure that the opponent pool slowed but did not prevent. CNN-DQN 5B instead passed through a noisy middle period while the pool filled with stronger snapshots, then climbed steadily to about 77% by episode 1500 and was still improving when training stopped.

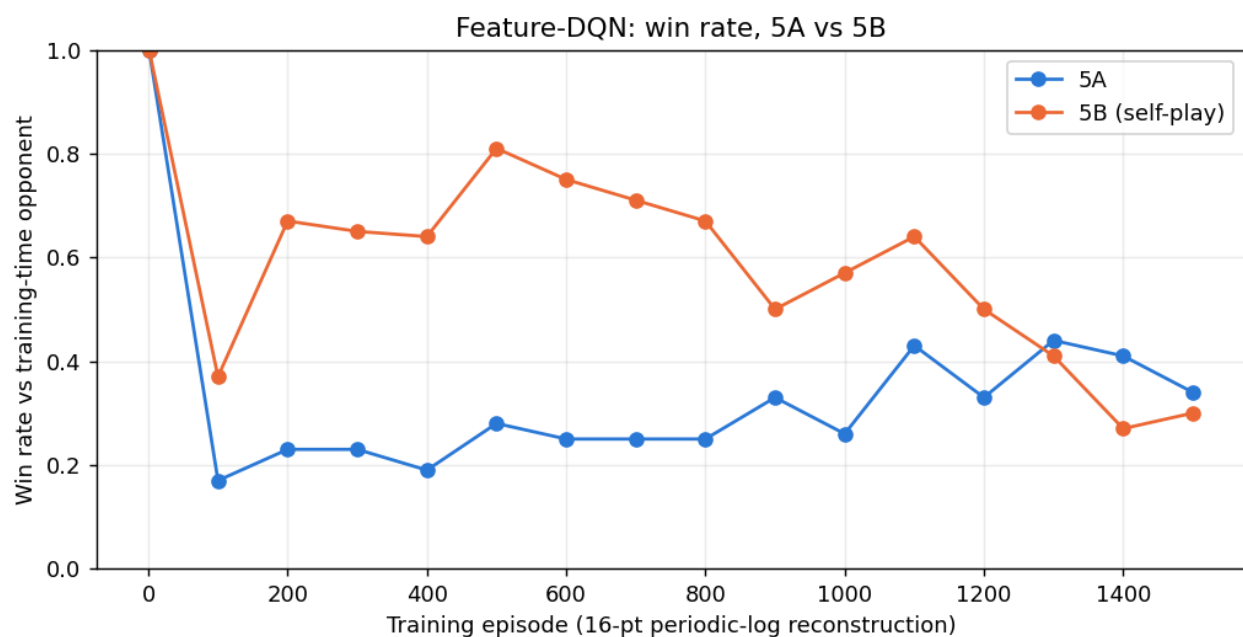


Figure 15. Feature-DQN Level 5 training-time win rate: fixed-opponent 5A vs self-play 5B.

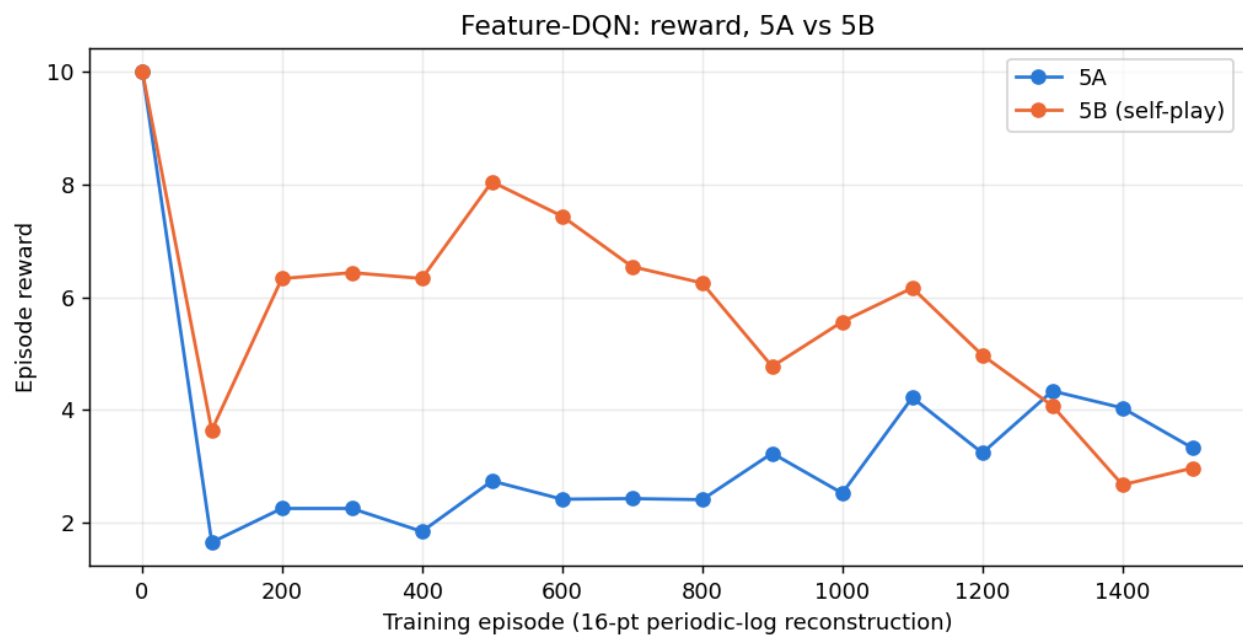


Figure 16. Feature-DQN Level 5 episode reward: fixed-opponent 5A vs self-play 5B.

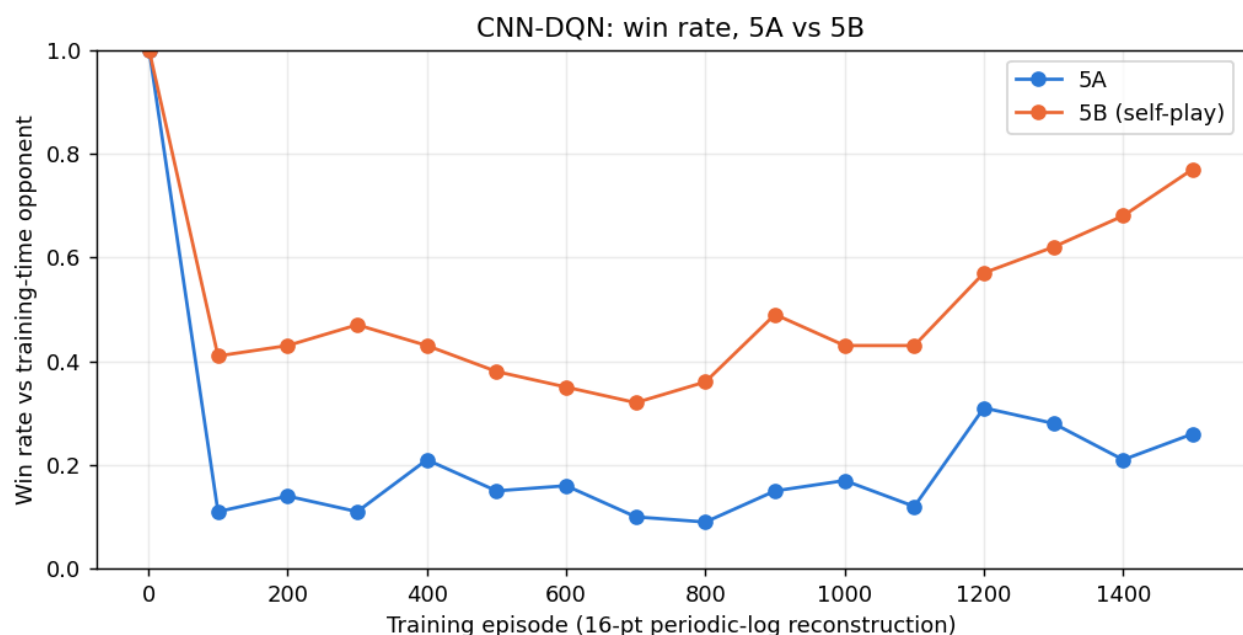


Figure 17. CNN-DQN Level 5 training-time win rate: fixed-opponent 5A vs self-play 5B.

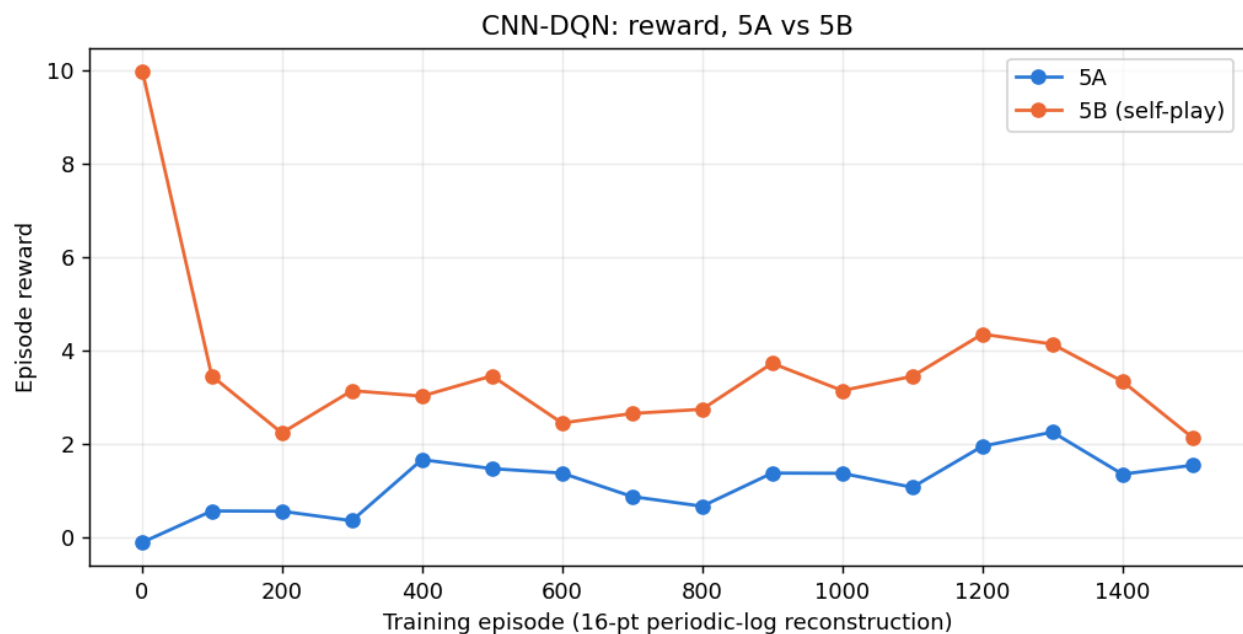


Figure 18. CNN-DQN Level 5 episode reward: fixed-opponent 5A vs self-play 5B.

The representation-capacity story from Level 4 reappears as a stability story in Level 5. CNN-DQN 5B beats frozen Level 4 72% of the time versus Feature-DQN 5B at 24%. However, Feature-DQN 5A initially beat its frozen Level 4 opponent more convincingly than CNN-DQN 5A (54% vs 20%), so the result is specifically about how each architecture responds to self-play rather than a blanket claim that CNN is always superior.

Tracking remains the hardest benchmark. Feature-DQN 5A and 5B each win only 2%; CNN-DQN rises from 2% in 5A to 20% in 5B. Thus self-play substantially improves CNN-DQN against frozen Level 4 and Random and helps against Tracking, but it does not solve Tracking-quality scripted play.

7.2 Replay evidence

Feature-DQN 5B vs frozen L4 (seed 22): left actions {UP:92, DOWN:38, STAY:74}; the frozen opponent stays still for 204 actions and Feature-DQN loses (-0.07 vs +10.06). CNN-DQN 5B vs frozen L4: left {UP:96, DOWN:68, STAY:36}, right {UP:77, DOWN:79, STAY:44}; the rally is actively contested and CNN-DQN wins (+9.96 vs -0.10).

8. Cross-level comparison and hyperparameter findings

Level / experiment	Primary question	Main finding
Level 1	Which tabular TD method behaves best?	Q-learning highest mean reward; Expected SARSA most hits/lowest variance; no strong solver.
Level 2A	Q-learning or SARSA vs scripted opponent?	Q-learning 21.5% vs SARSA 18.5% win rate.
Level 2B	Which epsilon schedule?	Linear decay led tested schedules at 20.5% win rate.
Level 3	Does zoo-trained Expected SARSA generalize?	Generalizes, but strong/tracking expose a large robustness gap.
Level 4	Do learned visual features help?	CNN-DQN 70% vs Feature-DQN 20.5%.
Level 5	Fixed opponent or self-play?	Architecture-dependent: CNN-DQN benefits strongly; Feature-DQN regresses.

The controlled design axes actually tested were TD target, exploration schedule, observation representation and self-play versus fixed-opponent training. A systematic learning-rate, discount-factor or network-width/depth sweep was not completed. The Level 5 live-mirror fraction of 0.8 was fixed a priori rather than swept.

9. Limitations and future work

- Levels 1-3 rely on one training seed and one evaluation seed set. Repeating 3-5 independent seeds and reporting mean +/- standard deviation would materially strengthen the tabular conclusions.
- Level 1 movement-penalty masking is a diagnostic reward-shaping ablation rather than a strictly unchanged official-reward training run. A direct official-reward rerun should be reported alongside it.
- Tabular state coverage remains sparse despite tens of thousands of learned states, contributing to weak values and opponent sensitivity.
- CNN-DQN is substantially more expensive computationally and still moves frequently. Movement-aware or anti-chattering tuning could improve efficiency without sacrificing win rate.
- The Level 4 neural comparison equalized episodes rather than total environment interactions; CNN-DQN accumulated more steps because its rallies lasted longer.
- Feature-DQN self-play was stopped at the planned 1,500 episodes during a decline. Extending training would distinguish a longer oscillation from a persistent instability.
- Level 5 full-resolution hits, steps and loss histories were not preserved in the periodic logs, so only captured reward/win-rate/epsilon curves are reported.
- The final competition checkpoint has not been tested end-to-end in the teaching team's competition notebook interface.

10. Final agents and checkpoints

Level	Final agent(s)	Checkpoint(s)
1	Q-learning, SARSA, Expected SARSA	q_learning.pkl; sarsa.pkl; expected_sarsa.pkl
2	Q-learning; best tested schedule: linear decay	q_learning_level2.pkl + schedule variants
3	Expected SARSA trained on opponent zoo	expected_sarsa_level3_zoo.pkl
4	Feature-DQN and CNN-DQN	feature_dqn_level4.pt; cnn_dqn_level4.pt
5	Feature-DQN and CNN-DQN, 5A and 5B variants	feature_dqn_level5_5a.pt / _selfplay.pt; cnn_dqn_level5_5a.pt / _selfplay.pt

The recommended competition checkpoint is `cnn_dqn_level5_selfplay.pt`. It is the strongest and most self-play-stable agent produced, with 72% win rate against frozen Level 4 and 98% against Random in the Level 5 evaluation. The important caveat is its 20% win rate against Tracking, so it should be described as the strongest submitted agent rather than as a solved policy.

11. Conclusion

Across five levels, the clearest pattern is that representation capacity governs both raw performance and training stability. Tabular methods establish useful algorithmic and exploration baselines but remain fragile against stronger opponents. CNN-DQN requires substantially more compute than Feature-DQN, yet at Level 4 it learns a far stronger pixel-based policy, and at Level 5 the same capacity advantage appears as stable improvement under self-play. Feature-DQN, by contrast, peaks early and regresses under the changing self-play opponent distribution. The opponent pool prevents complete degeneracy but does not fully solve non-stationarity. Future work should prioritize multi-seed evaluation, targeted hyperparameter tuning, stronger worst-case robustness and end-to-end competition validation.